Enterprise AI Knowledge Hub

ABSTRACT

Enterprise AI Knowledge Hub is a free, open-source retrieval-augmented generation (RAG) system that enables users to upload PDFs and ask questions through a web interface. It combines a FastAPI backend and a Streamlit frontend with local embeddings and reranking via sentence-transformers, open-source vector storage using Qdrant in local mode, and local LLM inference through Ollama, eliminating paid API dependencies. The pipeline covers PDF extraction, chunking, vector indexing, semantic retrieval, and grounded answer generation, exposed through REST endpoints and a simple UI. The system is designed for self-hosted deployment with SQLite by default, supports migrations, and includes security guidance for secrets and CORS in production.

INTRODUCTION

Enterprise AI Knowledge Hub is a free, open-source Retrieval-Augmented Generation (RAG) platform designed to help organizations extract knowledge from their internal documents quickly and securely. Modern enterprises produce vast amounts of PDFs—policies, manuals, reports, onboarding guides, and compliance documents—yet employees often struggle to find precise answers without reading entire files. This project addresses that gap by combining document ingestion, semantic search, and local large-language-model (LLM) inference into a single, end-to-end system. Users can upload PDFs through a simple web interface, ask natural-language questions, and receive grounded answers that are drawn directly from the uploaded content.

The core motivation behind the project is to make enterprise-grade document intelligence available without expensive paid APIs. Many RAG demos rely on commercial LLMs or hosted vector databases, which can introduce cost and privacy barriers. Enterprise AI Knowledge Hub is built to run fully on local or self-hosted infrastructure. It uses open-source components such as Fast API for the backend API, Streamlit for the frontend dashboard, sentence-transformers for embeddings and reranking, Qdrant in local mode for vector storage, and Ollama for local LLM inference. This stack enables a complete solution that can be deployed on a developer machine, a small server, or a private VM, while still delivering an interactive user experience.

PROBLEM STATEMENT

Organizations generate and store a large volume of internal documents—policies, SOPs, technical manuals, reports, onboarding guides, and compliance files—most of which are distributed as PDFs. Although this information is critical for daily operations, it is often difficult to access quickly because traditional keyword search is limited and manual reading is time-consuming. Employees and teams spend significant time searching for answers, leading to delays, inconsistent decisions, and repeated effort. This creates a clear need for a system that can understand document context, retrieve relevant information, and deliver concise answers on demand.

At the same time, many enterprises face constraints that make adoption of cloud-based AI services difficult. These include data privacy concerns, regulatory requirements, budget limitations, and restrictions on sending internal documents to external APIs. As a result, organizations need a local-first, cost-effective solution that can run on their own infrastructure while still delivering the benefits of modern AI-powered semantic search and question answering.

The core problem addressed by this project is the lack of an affordable, privacy-preserving, end-to-end platform that enables users to upload internal documents and ask natural-language questions with reliable, context-grounded answers. Existing solutions often require paid APIs, vendor lock-in, or complex deployment stacks. Many open-source alternatives focus only on isolated components (such as embeddings or vector search) rather than providing a complete, usable system with a frontend, API, and workflow that non-technical users can operate.

OBJECTIVES

- Enable reliable PDF ingestion, text extraction, and chunking.
- Create semantic embeddings for document chunks and store them in a local vector database.
- Retrieve and reranked the most relevant passages for a user's question.
- Generate concise answers using a local LLM (via Ollama) grounded in retrieved context.
- Provide a FastAPI backend with clean endpoints for upload, query, and health checks.
- Deliver a Streamlit dashboard for interactive use and demos.
- Ensure easy local deployment with minimal setup and clear security guidance.

LITERATURE REVIEW

Retrieval-Augmented Generation (RAG) emerged to address a core limitation of large language models: their knowledge is fixed at training time and can be hard to update or verify. The original RAG paper formalized a hybrid architecture that retrieves relevant passages from an external corpus and conditions generation on that evidence, improving factuality and provenance for knowledge-intensive tasks. This framing became a foundation for modern enterprise document QA systems. (arxiv.org)

Before RAG, retrieval-augmented pretraining demonstrated that language models could be strengthened by integrating a learned retriever during pretraining, enabling models to pull relevant documents at inference time. REALM showed how retrieval can be trained end-to-end and used across pretraining, fine-tuning, and inference, establishing a key pathway toward modular, interpretable knowledge access. (arxiv.org)

A critical enabler of RAG is strong neural retrieval. Dense Passage Retrieval (DPR) introduced a dual-encoder approach that learns dense representations for questions and passages, outperforming classical sparse methods like BM25 on open-domain QA benchmarks and enabling efficient top-k retrieval with vector similarity. DPR is widely used as a reference point for dense retrieval in RAG systems. (arxiv.org)

The RAG framework itself combines pretrained sequence-to-sequence generation with dense retrieval over a large text index, providing two formulations: conditioning on a fixed set of passages for the full output or allowing token-level retrieval. The paper showed improvements on multiple QA and generation tasks versus parametric-only baselines, establishing the practical value of retrieval-augmented generation. (arxiv.org)

For embedding and semantic search, Sentence-BERT (SBERT) is widely cited for producing meaningful sentence embeddings with efficient similarity search. SBERT's Siamese architecture reduces the computational cost of semantic retrieval while maintaining high accuracy on sentence similarity tasks, which directly supports the embedding stage in RAG pipelines. (aclanthology.org)

In production systems, vector databases are used to index and query embeddings at scale. Qdrant provides both server and local modes, enabling fast similarity search and local-first deployment without running a separate service—an important fit for privacy-sensitive or cost-constrained environments. (qdrant. tech)

Finally, recent surveys synthesize RAG techniques, evaluation metrics, and open challenges, highlighting ongoing gaps such as robust evaluation, retrieval-generation alignment, and grounding consistency across domains. These surveys validate RAG as a dominant paradigm while emphasizing the need for careful system design and benchmarking. (mdpi.com)

METHODOLOGY

1. **Problem framing and configuration** the system is defined as a local-first RAG platform with no paid APIs. All key settings (models, retrieval size, thresholds, context length) are controlled through. env, and FastAPI exposes health/debug endpoints to show active configuration.

2. **Document ingestion and extraction** uploads support PDF, DOCX, and TXT. PDFs are extracted in stages: PyMuPDF first, then pdfplumber if text is sparse, and OCR (Tesseract) if the file is image-based. DOCX extraction includes both paragraphs and table cells; TXT is read directly.

3. **Text cleaning and normalization** extracted text is cleaned to remove PDF artifacts: hyphenated line breaks are fixed, punctuation is normalized, headers/footers and page numbers are stripped, and whitespace is normalized.

4. **Chunking and metadata enrichment** clean text is split into overlapping semantic chunks using a recursive splitter with sentence-friendly separators. Very short chunks are filtered out, and each chunk receives metadata (source, page, unique chunk_id) for traceability and deduplication.

5. **Embedding and indexing** chunks are embedded locally with Sentence Transformers (BAAI/bge-large-en-v1.5) and stored in a local Qdrant vector DB. The system handles Qdrant lock conflicts and falls back to an in-memory store if Qdrant fails.

6. **Query expansion and retrieval** user questions are expanded into multiple phrasings via Ollama to improve recall.

Each query variant retrieves candidates from the vector store, and duplicates are removed while preserving correct metadata.

7. **Reranking and context assembly** a local cross-encoder reranked (ms-marco-MiniLM-L-12-v2) scores candidates and selects top-k chunks. If reranking fails, the pipeline falls back to unranked retrieval results. Chunks are concatenated into a context block, and source metadata is compiled.

8. **Answer Generation with Grounding and Fallbacks** Ollama generates answers using a strict grounding prompt. If context is insufficient, the model must say so. The response always includes a “Sources” section. The system handles missing models, GPU failures, or low memory by switching to CPU or smaller models.

9. **API + UI Delivery and Reliability** FastAPI exposes upload, ask, health, and debug endpoints. Streamlit provides the dashboard with upload, chat history, answers, sources, and context preview. The system includes robust error handling for empty inputs, extraction failures, and missing context.

SYSTEM REQUIREMENTS

Operating System

1. Windows 10/11, macOS, or Linux (64-bit)

Software

1. Python 3.10+ (recommended for compatibility with listed deps)
2. pip and venv
3. Ollama installed and running (ollama serve)
4. Tesseract OCR (optional, only needed for scanned/image PDFs)
5. Git (optional, for cloning/versioning)

Hardware (Minimum)

1. CPU: 4-core modern CPU
2. RAM: 8 GB (works but may be slow for larger models)
3. Disk: 5–10 GB free (Python env + models + vector DB)

Hardware (Recommended)

1. CPU: 8-core or better
2. RAM: 16–32 GB (for smoother local LLM + embeddings)
3. Disk: 20+ GB free (larger models + document stores)
4. GPU: Optional (NVIDIA GPU with CUDA helps, but CPU-only is supported)

Network / Ports

1. Localhost only; no external APIs required
2. Backend: http://127.0.0.1:8000
3. Streamlit UI: http://127.0.0.1:8501

Libraries (Installed via requirements.txt)

1. FastAPI + Uvicorn
2. Streamlit
3. Sentence-Transformers + Torch
4. Qdrant client (local mode)
5. LangChain text splitters (optional, fallback exists)
6. PDF/DOCX extraction: PyMuPDF, pdfplumber, PyPDF2, python-docx, Pillow, pytesseract

ADVANTAGES

1. **Fully local and free**: No paid APIs or cloud dependencies; privacy-friendly.
2. **End-to-end pipeline**: Upload → extract → chunk → embed → retrieve → generate in one system.
3. **Supports multiple formats**: PDF, DOCX, TXT with OCR fallback for scanned PDFs.
4. **Grounded answers**: Responses are tied to retrieved document content with sources.
5. **Modular architecture**: Easy to replace models or components (embedding, reranker, LLM).
6. **Simple UI + API**: Streamlit dashboard plus FastAPI endpoints.
7. **Scalable storage**: Qdrant local vector store handles growing document sets.

DISADVANTAGES

1. Local compute limits: Large models require high RAM/CPU; performance can be slow on low-end hardware.
2. No cloud-scale features: Lacks multi-user auth, role control, or distributed scaling.
3. OCR dependency: Scanned PDFs require Tesseract; quality varies.
4. Answer quality depends on retrieval: If relevant chunks aren't retrieved, answers may be incomplete.
5. Basic evaluation: No built-in benchmarking or accuracy metrics pipeline.
6. Latency: Local LLM inference is slower than hosted GPU services.
7. Limited file types: Doesn't yet support formats like PPTX, HTML, or images.

CONCLUSION

Enterprise AI Knowledge Hub demonstrates a complete, local-first RAG solution for enterprise documents. It integrates document ingestion, semantic retrieval, reranking, and grounded answer generation into a usable web application with both API and UI access. By relying on open-source tools (FastAPI, Streamlit, Sentence-Transformers, Qdrant, and Ollama), the project delivers privacy-preserving, cost-free document intelligence that can run entirely on local infrastructure. While it is not yet a full enterprise product in terms of scaling, governance, and evaluation, it provides a strong, end-to-end foundation that is practical, extensible, and recruiter-ready.